

TransformerEncoder#

<https://pytorch.org/docs/stable/generated/torch.nn.TransformerEncoder.html>

Description:

`TransformerEncoder` is a stack of `N` encoder layers. This `TransformerEncoder` layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem. All layers in the `TransformerEncoder` are initialized with the same parameters. It is recommended to manually initialize the layers after creating the `TransformerEncoder` instance. `encoder_layer` (`TransformerEncoderLayer`) – an instance of the `TransformerEncoderLayer()` class (required).

Function Signature

```
class torch.nn.TransformerEncoder(encoder_layer, num_layers,  
norm=None, enable_nested_tensor=True, mask_check=True)  
[source]#
```

Parameters

```
forward(src, mask=None, src_key_padding_mask=None,  
is_causal=None)[source]#
```

Parameters

Return type

Returns:

Tensor

Code Examples

```
>>> encoder_layer = nn.TransformerEncoderLayer(d_model=512,  
nhead=8)  
>>> transformer_encoder = nn.TransformerEncoder(encoder_layer,  
num_layers=6)  
>>> src = torch.rand(10, 32, 512)  
>>> out = transformer_encoder(src)
```

Notes & Warnings

WARNING

Warning All layers in the TransformerEncoder are initialized with the same parameters. It is recommended to manually initialize the layers after creating the TransformerEncoder instance.

Detailed Documentation

Documentation

TransformerEncoder is a stack of N encoder layers.

This TransformerEncoder layer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

All layers in the TransformerEncoder are initialized with the same parameters. It is recommended to manually initialize the layers after creating the TransformerEncoder instance.

`encoder_layer` (TransformerEncoderLayer) – an instance of the TransformerEncoderLayer() class (required).

`num_layers` (int) – the number of sub-encoder-layers in the encoder (required).

`norm` (Optional[Module]) – the layer normalization component (optional).

`enable_nested_tensor` (bool) – if True, input will automatically convert to nested tensor (and convert back on output). This will improve the overall performance of `TransformerEncoder` when padding rate is high. Default: True (enabled).

Pass the input through the encoder layers in turn.

`src` (Tensor) – the sequence to the encoder (required).

`mask` (Optional[Tensor]) – the mask for the src sequence (optional).

`src_key_padding_mask` (Optional[Tensor]) – the mask for the src keys per batch (optional).

`is_causal` (Optional[bool]) – If specified, applies a causal mask as mask. Default: None; try to detect a causal mask. Warning: `is_causal` provides a hint that mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

see the docs in `Transformer`.